MegaMOO

A MOO engine with Python as the in-world language · post for the Evennia community

Long-time lurker, first show-and-tell. This started here, so it seemed right to post it here.

A while back I built a full MOO platform on top of Evennia — the object model, the verb dispatch, the in-world programming, the lot. It worked, and I learned an enormous amount doing it. What I kept running into was that I was fighting the grain: the thing I wanted was objects parented to other *objects*, editable from inside the game, and Evennia's inheritance lives in typeclasses — Python classes on disk. (Prototypes are for spawning, which is a different job.) That's not a flaw in Evennia, it's a deliberate and sensible design. It just isn't the MOO one, and I wanted the MOO one badly enough to stop working around it.

So I wrote an engine. MegaMOO — LambdaMOO's object model, with Python as the in-world language instead of the MOO language:

```
pip install megamoo
megamoo init mygame
cd mygame && megamoo --dev
```

https://malifaxlax.github.io/megamoo/

That last command prints a URL. Opening it puts you in the world — the browser client ships with the server, so there is nothing else to install and nothing to point at it.

What it's good at, specifically:

- **Objects inherit from objects.** #17 is the parent of your rooms and you can edit it live, from inside the game, and every room changes.
- **Verbs are Python files.** One per verb. Edit in your editor and the running server picks it up; edit in-game with @program and the file is written for you. No reload, no restart.
- **Zero dependencies.** Standard library only — no Django, no ORM, no migrations. That's a real trade: I gave up everything Django gives you to get a server that's one process and one file.
- **A browser client that isn't an afterthought.** Served by the game itself over the same world telnet players are in. It lays text out on a terminal's character cell, so ASCII art and box drawing arrive the shape they were drawn rather than stretched into prose spacing. There's an automap built from the room graph, and a world can drop a splash image next to its login banner and the browser shows that instead.
- **SQLite object database**, TLS, MCCP2, MSSP.

To be clear about what this isn't: it's 0.10 beta, it's one person, and Evennia is vastly more capable and better supported. If you want the web integration, the batteries, the contribs, the community — that's Evennia and it isn't close. This is a narrower thing for people who specifically want the MOO model and would rather write Python than MOO code.

One thing worth knowing before you build something you care about. `megamoo init` hands you a copy of the starter world, and from that moment it is yours — your verbs, your rooms, your edits, and I never touch it again. The other side of that bargain is that improvements I make to the starter later do not find their own way into a world that already exists. Verb files you can carry across by hand, and the guide has a section on doing it; changes that live inside the world database do not have a path yet.

That is a 0.10 beta still growing into its shape rather than a decision I am defending, and it is moving: beta13 started recording which release a world was built from, which is the piece that turns "is this yours or just the old default?" into something answerable instead of a guess. If you are kicking the tyres it will never come up. If you are starting something long-lived, better to hear it from me now than discover it in a month.

Mostly I wanted to say thanks. Griatch, your work is why I understood what a MUD engine even looks like from the inside, and the reason I thought building one was a thing a person could do. A lot of MegaMOO's shape is me having internalised decisions from Evennia and then making different ones on purpose.

Happy to answer anything.